Chapter 12

RMI and CORBA

CHAPTER OUTLINES

To discuss basic concepts of applets

- To understand role of RMI and CORBA in creating distributed applications
- To explain RMI architecture
- To demonstrate steps involved in creating RMI applications
- To discuss basics of CORBA and compare it with RMI
- To understand architecture of CORBA

12.1 Introduction to RMI

The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM. The RMI provides remote communication between the applications using two objects *stub* and *skeleton*. RMI applications often comprise two separate programs, a server and a client. A typical server program creates some remote objects, makes references to these objects accessible, and waits for clients to invoke methods on these objects. A typical client program obtains a remote reference to one or more remote objects on a server and then invokes methods on them. RMI provides the mechanism by which the server and the client communicate and pass information back and forth.

A primary goal for the RMI designers was to allow programmers to develop distributed Java programs with the same syntax and semantics used for non-distributed programs. To do this, they had to carefully map how Java classes and objects work in a single Java Virtual Machine (JVM) to a new model of how classes and objects would work in a distributed (multiple JVM) computing environment.

12.2 Interfaces: The Heart of RMI

The RMI architecture is based on one important principle: the definition of behavior and the implementation of that behavior are separate concepts. RMI allows the code that defines the behavior and the code that implements the behavior to remain separate and to run on separate JVMs. This fits nicely with the needs of a distributed system where clients are concerned about the definition of a service and servers are focused on providing the service. Specifically, in RMI, the definition of a remote service is coded using a Java interface. The implementation of the remote service is coded in a class. Therefore, the key to understanding RMI is to remember that *interfaces define behavior* and *classes define implementation*. While the following diagram illustrates this separation.

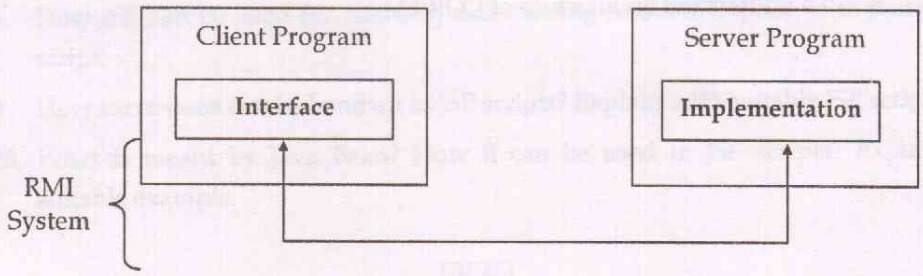

Figure 12.1 RMI interface and class.

Remember that a Java interface does not contain executable code. RMI supports two classes that implement the same interface. The first class is the implementation of the behavior, and it runs on the server. The second class acts as a proxy for the remote service and it runs on the client. This is shown in the following diagram.

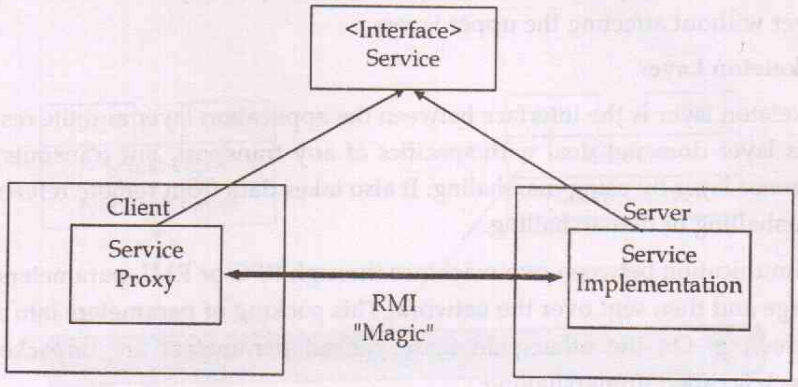

Figure 12.2 RMI proxy and implementation.

A client program makes method calls on the proxy object, RMI sends the request to the remote JVM, and forwards it to the implementation. Any return values provided by the implementation are sent back to the proxy and then to the client's program.

12.3 RMI Architecture Layers

The RMI system consists of three layers:

→ The stub/skeleton layer

→ The remote reference layer

→ The transport layer

The application layer sits on top of the RMI system.

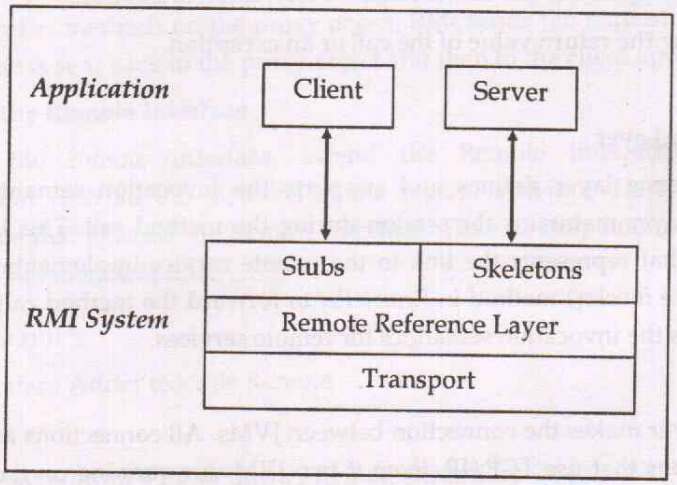

Figure 12.3 RMI architecture.

By using a layered architecture each of the layers could be enhanced or replaced without affecting the rest of the system. For example, the transport layer could be replaced by a UDP/IP layer without affecting the upper layers.

Stub and Skeleton Layer

The stub/skeleton layer is the interface between the application layer and the rest of the RMI system. This layer does not deal with specifics of any transport, but transmits data to the remote reference layer by using marshaling. It also takes data from remote reference layer by using unmarshalling or demarshalling.

During communication between two machines through RPC or RMI, parameters are packed into a message and then sent over the network. This packing of parameters into a message is called marshalling. On the other side these packed parameters are unpacked from the message which is called unmarshalling.

A *stub* for a remote object is the client-side proxy for the remote object. Such a stub implements all the interfaces that are supported by the remote object implementation. A client-side stub is responsible for:

→ Initiating a call to the remote object

→ Marshaling arguments

→ Informing the remote reference layer that the call should be invoked

→ Unmarshaling the return value or exception from a marshal stream

→ Informing the remote reference layer that the call is complete.

A *skeleton* for a remote object is a server-side entity that contains a method which dispatches calls to the actual remote object implementation. The skeleton is responsible for:

→ Unmarshaling arguments.

→ Making the up-call to the actual remote object implementation.

→ Marshaling the return value of the call or an exception.

Remote Reference Layer

The remote reference layer defines and supports the invocation semantics of the RMI connection. This layer maintains the session during the method call. This layer provides a *RemoteRef* object that represents the link to the remote service implementation object. The stub objects use the *invoke()* method in *RemoteRef* to forward the method call. The *RemoteRef* object understands the invocation semantics for remote services.

Transport Layer

The Transport Layer makes the connection between JVMs. All connections are stream-based network connections that use TCP/IP. Even if two JVMs are running on the same physical computer, they connect through their host computer's TCP/IP network protocol stack. (This is why you must have an operational TCP/IP configuration on your computer to run a RMI

programs). The following diagram shows the unfettered use of TCP/IP connections between JVMs.

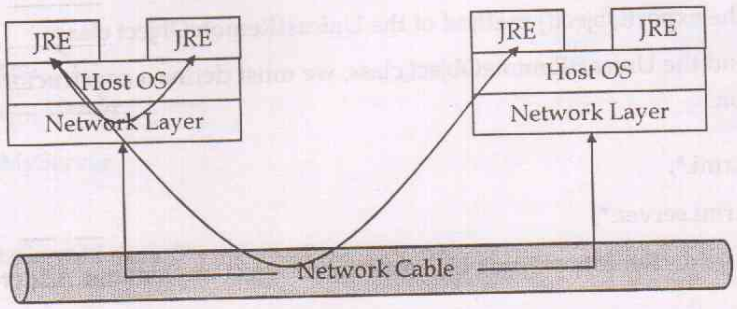

Figure 12.4 Role of Network layer in RMI.

12.4 Writing RMI Programs

There are six steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

The client application needs only two files, remote interface and client application. In the RMI application, both client and server interact with the remote interface. The client application invokes methods on the proxy object; RMI sends the request to the remote JVM. The return value is sent back to the proxy object and then to the client application.

Step 1: Create the Remote Interface

For creating the remote interface, extend the Remote interface and declare the RemoteException with all the methods of the remote interface. Here, we are creating a remote interface that extends the Remote interface. There is only one method named add() and it declares RemoteException.

```
import java.rmi.*;
public interface Adder extends Remote
{
    public int add(int x,int y) throws RemoteException;
}
```

Step 2: Provide the Implementation of the Remote Interface

Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to

→ either extend the UnicastRemoteObject class,

→ or use the exportObject() method of the UnicastRemoteObject class

In case, we extend the UnicastRemoteObject class, we must define a constructor that declares RemoteException.

```
import java.rmi.*;
import java.rmi.server.*;
public class AdderRemote extends UnicastRemoteObject implements Adder
{
    AdderRemote()throws RemoteException
    {
        super();
    }
    public int add(int x,int y)
    {
        return x+y;
    }
}
```

Step 3 :Create the Stub and Skeleton Objects Using the rmic Tool

Next step is to create stub and skeleton objects using the rmi compiler. The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

rmic AdderRemote

Step 4: Start the Registry Service by the rmiregistry Tool

Now start the registry service by using the rmiregistry tool. If we don't specify the port number, it uses a default port number. In this example, we are using the port number 5000.

rmiregistry 5000

Step 5: Create and Run the Server Application

Now rmi services need to be hosted in a server process. The Naming class provides methods to get and store the remote object. In this example, we are binding the remote object by the name rmiserver.

```
import java.rmi.*;
import java.rmi.registry.*;
public class MyServer
{
    public static void main(String args[ ])
    {
        try
        {
            Adder adderobject=new AdderRemote();
            Naming.rebind("rmi://localhost:5000/rmiserver", adderobject);
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Step 6: Create and run the client application

At the client we are getting the *adderobject* object by the lookup() method of the Naming class and invoking the method on this object. In this example, we are running the server and client applications, in the same machine so we are using localhost. If we want to access the remote object from another machine, change the localhost to the host name (or IP address) where the remote object is located.

```
import java.rmi.*;
public class MyClient{
public static void main(String args[]) throws Exception
{
Adder a=(Adder)Naming.lookup("rmi://localhost:5000/as");
int s=a.add(34,4);
```

```
System.out.println("Sum="+s);
}
}
```

12.5 Parameters in RMI

We have seen that RMI supports method calls to remote objects. When these calls involve passing parameters or accepting a return value, how does RMI transfer these between JVMs? What semantics are used? Does RMI support pass-by-value or pass-by-reference? The answer depends on whether the parameters are primitive data types, objects, or remote objects.

12.6 An Introduction to CORBA

Common Object request broker is used for communication between distributed and heterogeneous objects that are written by in java language are any other language. CORBA is developed by Object Management Group (OMG). OMG adopted the CORBA standerd in 1990. This was follwed by many many revisions of CORBA architecture.

12.6.1 Comparison of RMI and CORBA

RMI and CORBA are both defined for the communication of remote objects in the middle tier of distributed applications. RMI enables a java object to remotely communicate with other java objects, CORBA, on the other hand, provides a Solution that is portable across multiple platforms. CORBA provides an environment for the communication of objects regardless of the programming languages they are written or the operating system that they are running on. We can use CORBA to enable interaction between legacy systems and new technology applications.

RMI applications are slower than CORBA applications. This is because RMI is based on java, which is interpreted language. The execution of interpreted language has interpretation overheads that are not present in compiled languages. These interpretation overheads include the excess time taken to interpret each line of code as compared to compilation, in which entire code is compiled in one instance. Therefore, CORBA applications are suitable for real-time systems since their execution time is less than that of RMI applications.

RMI passes parameters by value whereas CORBA passes parameters by reference. Here are the steps for implementing CORBA objects:

1. Write the interface that specifies how the object works using IDL, the *interface definition language*, for defining CORBA interfaces. IDL is a special language to specify interfaces in a language-neutral form.
2. Using the IDL compiler(s) for the target language(s), generate the needed stub and helper classes.

3. Add the implementation code for the server objects, using the language of your choice. (The skeleton created by the IDL compiler is only glue code. You still need to provide the actual implementation code for the server methods.) Compile the implementation code.
4. Write a server program that creates and registers the server objects. The most convenient method for registration is to use the CORBA *naming service*, a service that is similar to the rmiregistry.
5. Write a client program that locates the server objects and invokes services on them.
6. Start the naming service and the server program on the server and the client program on the client.

These steps are quite similar to the steps that you use to build distributed applications with RMI. There are two important differences:

- You can use any language with a CORBA binding to implement clients and servers.
- You use IDL to specify interfaces.

12.7 Architecture of CORBA

The CORBA architecture consists of following components:

- Object Request broker (ORB)
- Common Object Services
- Interface Definition Language (IDL)
- Internet Inter-ORB Protocol (IIOP)
- Basic Object Adapter (BOA)

12.7.1 Object Request Broker

The foundation of CORBA architecture is based on the Object Request Broker (ORB). The ORB is prime mechanism that acts as a middleware between the client and the server. It provides a heterogeneous environment in which a client can remotely communicate with the server irrespective of the hardware, the operating systems, and the software development language used.

The ORB helps establish connection with remote server. When a client wants to use the services of a server, it first needs reference to the object that is providing the service. Here, the ORB plays its role; the ORB resolves the client request for the object reference on behalf of the client and thereby enables the client and the server to establish the connectivity.

12.7.2 Common Object Services

The Common object services help the ORB in enabling object to interact in a heterogeneous environment. These services are responsible for tasks such as creating objects, monitoring access control and relocated objects. One of the important object services is the Naming service. This service enables a remote server object to register itself with the ORB so that a client can use its services.

12.7.3 Interface Definition languages

CORBA uses the concept of an interface Definition language to implement the services. IDL is a neutral programming language method of defining how a service is implemented. The syntax of this language is independent of any programming language. IDL is used to define an interface, which describes the methods that are available and the parameters that are required when a call is made for a remote object.

12.7.4 Internet Inter-ORB protocol (IIOP)

The CORBA standard specifies General Inter-ORB protocol (GIOIP) as a standard protocol that defines how ORBs communicate with component. IIOP is a TCP/IP implementation of GIOP. It is standard protocol for communication between ORBs on TCP/IP based networks.

Vendors can define their own proprietary protocols for their implementation of CORBA. However they must implement IIOP for their products to be CORBA compliant. This ensures interoperability. When there is more than one protocol, The ORB can negotiate as to which protocol to use.

Since the Internet uses TCP/IP and IIOP is defined for TCP/IP, a CORBA application can communicate over the internet.

12.7.5 Basic Object Adapter (BOA)

The ORB also provides services other than those required for object communication. These services are provided by object adapters and can range from user authentication to object activation to object persistence. Object adapters are part of the ORB. The BOA provides the basic functions required for accessing the services of an ORB. If we compare the ORB to a computer, The BOA can be compared to a shell program pf the UNIX operating system.

One of the most important services provided by the BOA is the server activation policy service. The server activation policy defines how a server component should be started and executed. For example, a server might be initialized once and many clients might access the same server object.

12.8 The functions of CORBA Applications

In a CORBA application, the client application requests the Naming service for reference to an object. A reference to a CORBA object is returned and passed to the stub. The client invokes the method through the stub references. The stub in turn passes request to the skeleton reference obtained through the Naming service. The functioning of a CORBA application is illustrated in the figure given below:

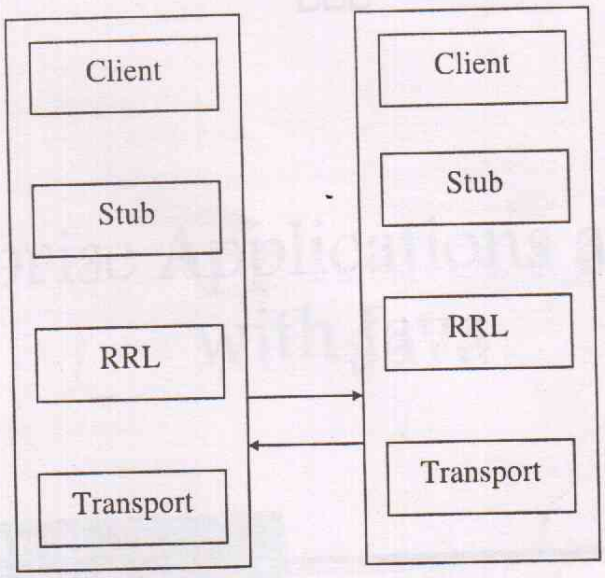

Figure 12.5: Functioning of CORBA application

The first step to design a CORBA application is to create an interface Definition language (IDL) file. We specify the functionality of CORBA application in this file. The IDl file contains the IDL interfaces, which when compiled generate the interface definition files, aserver application, The client stub, and the server skeleton. We need to save the IDl interface with the .idl extension.. The IDL interfaces are compiled using an IDl compiler.

Exercise

1. What is RMI? Discuss stub and skeleton. Explain its role in creating distributed applications.
2. Discuss RMI architecture with suitable diagram and discus each layers in the architecture briefly.
3. Explain the steps involved in creating RMI applications with suitable java code.

4. Write server and client program by using RMI such that the program finds factorial of n.
5. What is CORBA? Compare it with RMI. Discuss the steps used in creating RMI objects
6. Explain CORBA architecture briefly. Also discuss the functions of CORBA application.

❑❑❑